

Assignment: Debugging and Optimization in Machine Learning Algorithms

Q1. Perceptron Convergence Failure

Even on linearly separable data, a perceptron can fail to converge due to: (1) a learning rate that is too high, causing the weight vector to oscillate past the correct boundary, or too low, causing extremely slow movement; (2) poor weight initialization (e.g., very large random values) that puts the decision boundary far from any reasonable region; (3) bias term omitted or not updated, which prevents the boundary from shifting away from the origin; (4) an incorrect update rule (wrong sign, updating on correctly classified points, or forgetting to loop until zero errors). Debugging strategy: log the number of misclassifications per epoch — if it oscillates instead of shrinking, suspect the learning rate; if it stays flat, suspect the update rule or bias; print weights every epoch to check for divergence; verify the update is only triggered on misclassified samples. Optimization: use a moderate fixed or decaying learning rate, initialize weights to small random values, always include and update the bias, and add a maximum-epoch safeguard with an explicit convergence check.

```
import numpy as np

class Perceptron:
    def __init__(self, n_features, lr=0.1, epochs=1000):
        self.lr = lr
        self.epochs = epochs
        self.w = np.random.uniform(-0.01, 0.01, n_features) # small init, avoids divergence
        self.b = 0.0

    def predict(self, x):
        return 1 if np.dot(self.w, x) + self.b >= 0 else -1

    def fit(self, X, y, verbose=True):
        history = []
        for epoch in range(self.epochs):
            errors = 0
            for xi, yi in zip(X, y):
                pred = self.predict(xi)
                if pred != yi:
                    self.w += self.lr * yi * xi # update only on misclassified points
                    self.b += self.lr * yi # correct update rule
                    errors += 1 # bias must be updated too
            history.append(errors)
            if verbose:
                print(f"Epoch {epoch+1:3d} | misclassified = {errors} | ||w|| = {np.linalg.norm(self.w):.4f}")
            if errors == 0:
                print(f"Converged after {epoch+1} epochs.")
                break
        else:
            print("Did NOT converge within max epochs -- check lr / init / bias / update rule.")
        return history

# Debugging harness: sweep learning rates to see which ones oscillate vs converge
if __name__ == "__main__":
    np.random.seed(1)
    X = np.array([[2, 3], [1, 1], [4, 5], [6, 5], [1, 0], [3, 2], [5, 6], [0, 1]])
    y = np.array([1, -1, 1, 1, -1, -1, 1, -1])
    for lr in [5.0, 0.5, 0.05, 0.001]:
        print(f"\n--- Testing learning rate = {lr} ---")
        model = Perceptron(n_features=2, lr=lr, epochs=200)
        model.fit(X, y, verbose=False)
        preds = [model.predict(x) for x in X]
        acc = np.mean(np.array(preds) == y)
        print(f"lr={lr:<6} final accuracy={acc:.2f} final weights={model.w}, bias={model.b:.3f}")
```

Q2. Slow Convergence / Local Minima in Backpropagation

Slow convergence and local-minima traps in an MLP are usually caused by vanishing gradients (sigmoid/tanh activations saturate, so gradients shrink layer by layer), poor weight initialization (e.g., zeros, which makes all neurons symmetric, or very large values that saturate activations), a learning rate that is too small (slow) or too large (unstable), and lack of normalization causing uneven gradient scales. Debugging approach: print/plot the gradient norm per layer during training -- if gradients shrink toward the input layers, that confirms vanishing gradients; plot the loss curve -- a long flat plateau suggests a local minimum or saturated activations; check that weights are not initialized identically. Optimization: switch to ReLU/leaky-ReLU, use He/Xavier initialization, apply batch normalization, and use adaptive optimizers such as Adam with a learning-rate schedule.

```
import numpy as np

def relu(z):      return np.maximum(0, z)
def relu_deriv(z): return (z > 0).astype(float)
def sigmoid(z):   return 1 / (1 + np.exp(-z))
def sigmoid_deriv(a): return a * (1 - a)

class MLP:
    def __init__(self, sizes, lr=0.01, activation="relu"):
        self.sizes = sizes
        self.lr = lr
        self.activation = activation
        self.W, self.B = [], []
        for i in range(len(sizes) - 1):
            fan_in = sizes[i]
            # He initialization for ReLU, Xavier for sigmoid -- prevents vanishing/exploding grads
            scale = np.sqrt(2.0 / fan_in) if activation == "relu" else np.sqrt(1.0 / fan_in)
            self.W.append(np.random.randn(sizes[i], sizes[i+1]) * scale)
            self.B.append(np.zeros(sizes[i+1]))
        # Adam optimizer state
        self.mW = [np.zeros_like(w) for w in self.W]; self.vW = [np.zeros_like(w) for w in self.W]
        self.mB = [np.zeros_like(b) for b in self.B]; self.vB = [np.zeros_like(b) for b in self.B]
        self.t = 0

    def forward(self, X):
        A = [X]
        for i, (w, b) in enumerate(zip(self.W, self.B)):
            z = A[-1] @ w + b
            a = relu(z) if (i < len(self.W)-1 and self.activation == "relu") else sigmoid(z)
            A.append(a)
        return A

    def backward(self, A, y):
        grads_w, grads_b = [None]*len(self.W), [None]*len(self.B)
        delta = (A[-1] - y) * sigmoid_deriv(A[-1]) # output layer error
        grad_norms = []
        for l in reversed(range(len(self.W))):
            grads_w[l] = A[l].T @ delta / len(y)
            grads_b[l] = delta.mean(axis=0)
            grad_norms.append(np.linalg.norm(grads_w[l])) # track for vanishing-gradient debugging
            if l > 0:
                deriv = relu_deriv(A[l]) if self.activation == "relu" else sigmoid_deriv(A[l])
                delta = (delta @ self.W[l].T) * deriv
        return grads_w, grads_b, grad_norms[::-1]

    def adam_step(self, grads_w, grads_b, beta1=0.9, beta2=0.999, eps=1e-8):
        self.t += 1
        for i in range(len(self.W)):
            self.mW[i] = beta1*self.mW[i] + (1-beta1)*grads_w[i]
            self.vW[i] = beta2*self.vW[i] + (1-beta2)*(grads_w[i]**2)
            mW_hat = self.mW[i] / (1 - beta1**self.t); vW_hat = self.vW[i] / (1 - beta2**self.t)
            self.W[i] -= self.lr * mW_hat / (np.sqrt(vW_hat) + eps)

            self.mB[i] = beta1*self.mB[i] + (1-beta1)*grads_b[i]
            self.vB[i] = beta2*self.vB[i] + (1-beta2)*(grads_b[i]**2)
            mB_hat = self.mB[i] / (1 - beta1**self.t); vB_hat = self.vB[i] / (1 - beta2**self.t)
            self.B[i] -= self.lr * mB_hat / (np.sqrt(vB_hat) + eps)

    def train(self, X, y, epochs=500):
        for epoch in range(epochs):
            A = self.forward(X)
            loss = np.mean((A[-1] - y) ** 2)
```

```

        grads_w, grads_b, grad_norms = self.backward(A, y)
        self.adam_step(grads_w, grads_b)
        if epoch % 50 == 0:
            print(f"Epoch {epoch:4d} | loss={loss:.5f} | grad-norms per layer={[round(g,5) for g in grad_norms]}")

if __name__ == "__main__":
    np.random.seed(0)
    X = np.random.randn(200, 4)
    y = ((X[:, 0] + X[:, 1] - X[:, 2] > 0).astype(float)).reshape(-1, 1)
    net = MLP(sizes=[4, 16, 8, 1], lr=0.01, activation="relu")
    net.train(X, y, epochs=300)

```

Q3. Inconsistent Genetic Algorithm Solutions

Inconsistent GA results across runs usually stem from a poor chromosome encoding that does not map cleanly onto the search space, a fitness function that is noisy, mis-scaled, or does not discriminate between good and bad hypotheses, premature convergence caused by excessive selection pressure or too small a population, and low population diversity from insufficient mutation. Debugging strategy: plot best/average/worst fitness per generation across several seeds -- if average fitness collapses to best fitness very early, that indicates premature convergence; track population diversity (e.g., mean pairwise Hamming distance) each generation; verify the fitness function on hand-crafted known-good and known-bad chromosomes. Optimization: tune mutation rate adaptively, reduce selection pressure (e.g., tournament size), and add elitism so the best individual is never lost while still allowing exploration.

```

import random

class GeneticAlgorithm:
    def __init__(self, gene_length, pop_size=60, mutation_rate=0.02,
                 tournament_size=3, elitism=2, generations=150):
        self.gene_length = gene_length
        self.pop_size = pop_size
        self.mutation_rate = mutation_rate
        self.tournament_size = tournament_size # lower = less selection pressure = more diversity
        self.elitism = elitism # guarantees best solution is never lost
        self.generations = generations
        self.population = [self._random_chromosome() for _ in range(pop_size)]

    def _random_chromosome(self):
        return [random.randint(0, 1) for _ in range(self.gene_length)]

    def fitness(self, chromosome):
        # Example: maximize number of 1s while rewarding alternating structure (avoids trivial local optima)
        ones = sum(chromosome)
        alternation = sum(1 for i in range(len(chromosome)-1) if chromosome[i] != chromosome[i+1])
        return ones + 0.5 * alternation

    def tournament_select(self):
        contenders = random.sample(self.population, self.tournament_size)
        return max(contenders, key=self.fitness)

    def crossover(self, p1, p2):
        point = random.randint(1, self.gene_length - 1)
        return p1[:point] + p2[point:], p2[:point] + p1[point:]

    def mutate(self, chromosome):
        return [1 - g if random.random() < self.mutation_rate else g for g in chromosome]

    def diversity(self):
        # Mean pairwise Hamming distance -- debugging metric for population collapse
        n = len(self.population)
        total, count = 0, 0
        for i in range(min(n, 20)):
            for j in range(i+1, min(n, 20)):
                total += sum(a != b for a, b in zip(self.population[i], self.population[j]))
                count += 1
        return total / max(count, 1)

    def run(self):
        for gen in range(self.generations):
            self.population.sort(key=self.fitness, reverse=True)
            best_f = self.fitness(self.population[0])

```

```

    avg_f = sum(self.fitness(c) for c in self.population) / self.pop_size
    div = self.diversity()
    if gen % 10 == 0:
        print(f"Gen {gen:3d} | best={best_f:.2f} avg={avg_f:.2f} diversity={div:.2f}")

    new_pop = self.population[:self.elitism]      # elitism
    while len(new_pop) < self.pop_size:
        p1, p2 = self.tournament_select(), self.tournament_select()
        c1, c2 = self.crossover(p1, p2)
        new_pop.append(self.mutate(c1))
        if len(new_pop) < self.pop_size:
            new_pop.append(self.mutate(c2))
    self.population = new_pop
    self.population.sort(key=self.fitness, reverse=True)
    return self.population[0], self.fitness(self.population[0])

if __name__ == "__main__":
    random.seed(42)
    ga = GeneticAlgorithm(gene_length=20)
    best, best_fit = ga.run()
    print("Best chromosome:", best, "fitness:", best_fit)

```

Q4. Overfitting and High Cost in DNN + Genetic Programming

When a deep network combined with genetic programming overfits and is expensive to run, common causes are excessive model complexity (too many layers/parameters relative to data size), over-parameterized GP-evolved architectures, and insufficient training data or evaluation shortcuts that reward complexity. Debugging approach: plot training vs validation loss -- a widening gap confirms overfitting; log parameter counts and per-individual evaluation time in the GP population to find bloated architectures; use k-fold cross-validation to check that performance is not an artifact of one split. Optimization: add L2/L1 regularization and dropout, prune low-importance weights/connections, use early stopping, penalize model size directly inside the GP fitness function, and evaluate candidates on a small proxy dataset before full training to cut computational cost.

```

import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F

class RegularizedNet(nn.Module):
    def __init__(self, in_dim, hidden_dims, dropout=0.3, l2=1e-4):
        super().__init__()
        layers = []
        prev = in_dim
        for h in hidden_dims:
            layers += [nn.Linear(prev, h), nn.ReLU(), nn.Dropout(dropout)]
            prev = h
        layers.append(nn.Linear(prev, 1))
        self.net = nn.Sequential(*layers)
        self.l2 = l2

    def forward(self, x):
        return self.net(x)

def train_with_early_stopping(model, X_train, y_train, X_val, y_val,
                              epochs=300, patience=15, lr=1e-3):
    opt = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=model.l2)  # L2 regularization
    best_val, wait, best_state = float("inf"), 0, None
    for epoch in range(epochs):
        model.train()
        opt.zero_grad()
        pred = model(X_train)
        loss = F.mse_loss(pred, y_train)
        loss.backward()
        opt.step()

        model.eval()
        with torch.no_grad():
            val_loss = F.mse_loss(model(X_val), y_val).item()
        if val_loss < best_val - 1e-5:
            best_val, wait, best_state = val_loss, 0, {k: v.clone() for k, v in model.state_dict().items()}

```

```

    else:
        wait += 1
    if epoch % 20 == 0:
        print(f"Epoch {epoch:3d} | train={loss.item():.4f} | val={val_loss:.4f} | wait={wait}")
    if wait >= patience:
        # early stopping
        print(f"Early stopping at epoch {epoch}")
        break
    model.load_state_dict(best_state)
    return model

def prune_small_weights(model, threshold=1e-2):
    # Simple magnitude-based pruning to cut parameter count / compute cost
    pruned = 0
    with torch.no_grad():
        for p in model.parameters():
            mask = p.abs() < threshold
            pruned += mask.sum().item()
            p[mask] = 0.0
    print(f"Pruned {pruned} weights below threshold {threshold}")

def gp_fitness(individual_params, val_score, complexity_penalty=0.01):
    # Penalize architecture size directly inside the GP fitness function
    n_params = sum(p.numel() for p in individual_params)
    return val_score - complexity_penalty * np.log1p(n_params)

if __name__ == "__main__":
    torch.manual_seed(0)
    X_train = torch.randn(300, 10); y_train = torch.randn(300, 1)
    X_val = torch.randn(80, 10); y_val = torch.randn(80, 1)
    model = RegularizedNet(in_dim=10, hidden_dims=[64, 32], dropout=0.3, l2=1e-4)
    model = train_with_early_stopping(model, X_train, y_train, X_val, y_val)
    prune_small_weights(model, threshold=0.01)

```

Q5. Decision Tree Overfitting on Large Data

High training accuracy with poor test accuracy points to an overfit, overly deep tree that has memorized noise, or noisy/misabeled data being fit exactly. Debugging: plot training vs validation accuracy against tree depth to find where the validation curve turns down while training keeps rising; inspect leaf node sample counts -- many leaves with very few samples indicate over-splitting; run k-fold cross-validation to confirm the gap is systematic. Optimization: prune the tree (pre-pruning via `max_depth/min_samples_leaf` or post-pruning via cost-complexity pruning), use cross-validation to select hyperparameters, and apply feature selection to remove noisy, uninformative columns before training.

```

import numpy as np
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score, GridSearchCV
from sklearn.datasets import make_classification
from sklearn.feature_selection import SelectKBest, mutual_info_classif
from sklearn.pipeline import Pipeline

X, y = make_classification(n_samples=2000, n_features=30, n_informative=8,
                           n_redundant=10, n_repeated=2, flip_y=0.05, random_state=1)

# Step 1: debug -- show overfitting by scanning max_depth
print("Depth | Train Acc | CV Acc")
for depth in [None, 20, 12, 8, 5, 3]:
    clf = DecisionTreeClassifier(max_depth=depth, random_state=1)
    clf.fit(X, y)
    train_acc = clf.score(X, y)
    cv_acc = cross_val_score(clf, X, y, cv=5).mean()
    print(f"{str(depth):5} | {train_acc:.3f} | {cv_acc:.3f}")

# Step 2: feature selection to remove noisy features
selector = SelectKBest(mutual_info_classif, k=10)

# Step 3: grid search with cross-validation to pick pruning hyperparameters (cost-complexity pruning too)
pipe = Pipeline([("select", selector), ("tree", DecisionTreeClassifier(random_state=1))])
param_grid = {
    "tree__max_depth": [3, 5, 8, 12],
    "tree__min_samples_leaf": [1, 5, 10, 20],
    "tree__ccp_alpha": [0.0, 0.001, 0.01, 0.02], # cost-complexity pruning strength
}

```

```

grid = GridSearchCV(pipe, param_grid, cv=5, scoring="accuracy", n_jobs=-1)
grid.fit(X, y)
print("\nBest params:", grid.best_params_)
print("Best CV accuracy:", grid.best_score_)

best_model = grid.best_estimator_
print("Final training accuracy:", best_model.score(X, y))

```

Q6. Unstable CNN Training Across Epochs

Unstable accuracy/loss across epochs in a CNN is often caused by data augmentation that distorts images too aggressively (or inconsistently between train/val), vanishing gradients in very deep stacks without normalization, and incorrectly configured batch normalization (e.g., wrong axis, not switching between train/eval mode, or too small a batch size making batch statistics noisy). Debugging: plot the loss curve at a fine granularity (per batch, not just per epoch) to find abrupt spikes; disable augmentation temporarily to see whether instability persists; verify BatchNorm layers are set to eval() mode during validation and that running statistics are updating correctly. Optimization: use gradient clipping, tune batch size/learning rate together, add residual/skip connections, and use a learning-rate warmup with a scheduler for smoother convergence.

```

import torch
import torch.nn as nn
import torch.nn.functional as F

class StableCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, 3, padding=1)
        self.bn1 = nn.BatchNorm2d(32)
        self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
        self.bn2 = nn.BatchNorm2d(64)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(64 * 8 * 8, 128)
        self.fc2 = nn.Linear(128, num_classes)

    def forward(self, x):
        x = self.pool(F.relu(self.bn1(self.conv1(x))))
        x = self.pool(F.relu(self.bn2(self.conv2(x))))
        x = x.flatten(1)
        x = F.relu(self.fc1(x))
        return self.fc2(x)

def train_one_epoch(model, loader, optimizer, device, clip_value=1.0):
    model.train()
    losses = []
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        optimizer.zero_grad()
        out = model(x)
        loss = F.cross_entropy(out, y)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), clip_value) # prevents loss spikes
        optimizer.step()
        losses.append(loss.item())
    return losses # inspect per-batch losses to catch spikes, not just per-epoch average

def evaluate(model, loader, device):
    model.eval() # switches BatchNorm to running-stat mode
    correct, total = 0, 0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(device), y.to(device)
            preds = model(x).argmax(dim=1)
            correct += (preds == y).sum().item()
            total += y.size(0)
    return correct / total

if __name__ == "__main__":
    device = "cuda" if torch.cuda.is_available() else "cpu"
    model = StableCNN().to(device)
    optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
    scheduler = torch.optim.lr_scheduler.OneCycleLR(
        optimizer, max_lr=1e-3, total_steps=100, pct_start=0.1 # warmup then decay -- smooths training

```

```

)
dummy_loader = [(torch.randn(16, 3, 32, 32), torch.randint(0, 10, (16,))) for _ in range(20)]
for epoch in range(5):
    batch_losses = train_one_epoch(model, dummy_loader, optimizer, device)
    scheduler.step()
    print(f"Epoch {epoch} | mean loss={sum(batch_losses)/len(batch_losses):.4f} "
          f"| max spike={max(batch_losses):.4f}")

```

Q7. Reinforcement Learning Agent Fails to Learn

An RL agent that cannot find a good policy is usually suffering from poor reward design (sparse, delayed, or misaligned rewards that do not guide behaviour), insufficient exploration (the agent gets stuck exploiting an early mediocre policy), or an incorrect/incomplete state representation that hides information the agent needs. Debugging: log episode return over time -- flat, noisy returns suggest an exploration problem; log the entropy of the action distribution -- collapsing entropy early indicates premature exploitation; inspect the state features fed to the agent versus what the environment actually needs to be Markovian. Optimization: apply reward shaping to give denser feedback, use epsilon-decay or entropy-bonus exploration strategies, and tune hyperparameters (learning rate, discount factor, batch size) systematically via a small grid or random search.

```

import numpy as np

class GridWorld:
    def __init__(self, size=5, goal=(4, 4)):
        self.size = size
        self.goal = goal
        self.reset()

    def reset(self):
        self.pos = (0, 0)
        return self.pos

    def step(self, action):
        x, y = self.pos
        moves = {0: (-1, 0), 1: (1, 0), 2: (0, -1), 3: (0, 1)}
        dx, dy = moves[action]
        nx, ny = max(0, min(self.size-1, x+dx)), max(0, min(self.size-1, y+dy))
        self.pos = (nx, ny)
        # Reward shaping: dense negative distance-to-goal reward instead of sparse +1 at the end
        dist = abs(self.goal[0]-nx) + abs(self.goal[1]-ny)
        reward = -dist * 0.1
        done = self.pos == self.goal
        if done:
            reward += 10.0
        return self.pos, reward, done

class QLearningAgent:
    def __init__(self, n_states, n_actions, lr=0.1, gamma=0.95,
                 eps_start=1.0, eps_end=0.05, eps_decay=0.995):
        self.Q = np.zeros((n_states, n_actions))
        self.lr = lr
        self.gamma = gamma
        self.eps = eps_start
        self.eps_end = eps_end
        self.eps_decay = eps_decay

    def act(self, state_idx):
        if np.random.rand() < self.eps:
            # exploration -- decays over time
            return np.random.randint(self.Q.shape[1])
        return int(np.argmax(self.Q[state_idx]))

    def update(self, s, a, r, s_next, done):
        target = r if done else r + self.gamma * np.max(self.Q[s_next])
        self.Q[s, a] += self.lr * (target - self.Q[s, a])

    def decay_epsilon(self):
        self.eps = max(self.eps_end, self.eps * self.eps_decay)

def state_to_idx(pos, size):
    return pos[0] * size + pos[1]

```

```

if __name__ == "__main__":
    np.random.seed(0)
    size = 5
    env = GridWorld(size=size)
    agent = QLearningAgent(n_states=size*size, n_actions=4)

    episode_returns = []
    for episode in range(300):
        state = env.reset()
        total_r, done, steps = 0.0, False, 0
        while not done and steps < 100:
            s_idx = state_to_idx(state, size)
            action = agent.act(s_idx)
            next_state, reward, done = env.step(action)
            agent.update(s_idx, action, reward, state_to_idx(next_state, size), done)
            state = next_state
            total_r += reward
            steps += 1
        agent.decay_epsilon()
        episode_returns.append(total_r)
        if episode % 30 == 0:
            recent = np.mean(episode_returns[-30:])
            print(f"Episode {episode:3d} | return={total_r:6.2f} | avg(last30)={recent:6.2f} | eps={agent.eps:}

```

Q8. Inconsistent K-means Cluster Assignments

K-means giving different clusters on similar data is typically caused by random/poor centroid initialization (different starting points converge to different local optima), choosing the wrong number of clusters k , and sensitivity to outliers or unscaled features that distort distance calculations. Debugging: run K-means multiple times with different random seeds and compare inertia/cluster labels -- large variance across runs confirms an initialization problem; plot the elbow curve and silhouette score across a range of k values to check whether k is well chosen; visualize feature scales/outliers before clustering. Optimization: use K-means++ initialization for smarter starting centroids, normalize or standardize features first, remove or robustly handle outliers, and pick k via silhouette analysis rather than guessing.

```

import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
from sklearn.datasets import make_blobs

X, _ = make_blobs(n_samples=500, centers=4, cluster_std=1.4, random_state=3)
X[0:5] += 25 # inject artificial outliers to demonstrate sensitivity

# Step 1: debug -- compare plain random init vs k-means++ across several seeds
print("== Stability check: random init vs k-means++ ==")
for init in ["random", "k-means++"]:
    inertias = []
    for seed in range(10):
        km = KMeans(n_clusters=4, init=init, n_init=1, random_state=seed)
        km.fit(X)
        inertias.append(km.inertia_)
    print(f"init={init:10} | inertia mean={np.mean(inertias):.2f} std={np.std(inertias):.2f}")

# Step 2: scale features -- distance-based methods are sensitive to feature scale
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Step 3: robustly reduce outlier influence (simple clip-based approach)
q_low, q_high = np.percentile(X_scaled, [1, 99], axis=0)
X_clipped = np.clip(X_scaled, q_low, q_high)

# Step 4: choose k via silhouette analysis instead of guessing
print("\n== Choosing k via silhouette score ==")
best_k, best_score = None, -1
for k in range(2, 8):
    km = KMeans(n_clusters=k, init="k-means++", n_init=10, random_state=42)
    labels = km.fit_predict(X_clipped)
    score = silhouette_score(X_clipped, labels)
    print(f"k={k} | silhouette={score:.3f}")
    if score > best_score:
        best_k, best_score = k, score

```



```
print(f"\nBest k selected = {best_k} (silhouette={best_score:.3f})")
final_km = KMeans(n_clusters=best_k, init="k-means++", n_init=10, random_state=42)
final_labels = final_km.fit_predict(X_clipped)
```

Q9. SVM Underperformance on Nonlinear Data

An SVM that underperforms on nonlinear data is often using the wrong kernel (e.g., a linear kernel on data that needs a nonlinear boundary), poorly tuned hyperparameters (C too high causes overfitting, too low causes underfitting; gamma too high overfits locally, too low underfits), or unscaled features that distort the margin computation since SVMs are distance-based. Debugging: visualize the decision boundary on 2D projections to see if it is clearly too simple (linear) for the data shape; run a validation curve over C and gamma to see where performance peaks versus overfits; check feature scales before and after standardization. Optimization: switch to an RBF (or polynomial) kernel for nonlinear separability, run a grid search with cross-validation over C and gamma, and always standardize features beforehand.

```
import numpy as np
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.datasets import make_moons
from sklearn.pipeline import Pipeline

X, y = make_moons(n_samples=500, noise=0.25, random_state=7) # inherently nonlinear dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=7)

# Step 1: debug -- show that a linear kernel underperforms on nonlinear data
linear_svm = Pipeline([("scale", StandardScaler()), ("svc", SVC(kernel="linear"))])
linear_svm.fit(X_train, y_train)
print("Linear kernel test accuracy:", linear_svm.score(X_test, y_test))

# Step 2: try RBF kernel with default params
rbf_default = Pipeline([("scale", StandardScaler()), ("svc", SVC(kernel="rbf"))])
rbf_default.fit(X_train, y_train)
print("RBF (default C, gamma) test accuracy:", rbf_default.score(X_test, y_test))

# Step 3: grid search over C and gamma with cross-validation to find the right region
pipe = Pipeline([("scale", StandardScaler()), ("svc", SVC(kernel="rbf"))])
param_grid = {
    "svc_C": [0.1, 1, 10, 100],
    "svc_gamma": [0.001, 0.01, 0.1, 1, "scale"],
}
grid = GridSearchCV(pipe, param_grid, cv=5, scoring="accuracy", n_jobs=-1)
grid.fit(X_train, y_train)
print("\nBest params:", grid.best_params_)
print("Best CV accuracy:", grid.best_score_)
print("Test accuracy with tuned RBF SVM:", grid.score(X_test, y_test))
```

Q10. Poor Long-Term Dependency Learning in RNNs

Vanilla RNNs struggle with long-term dependencies mainly because of vanishing (and sometimes exploding) gradients during backpropagation-through-time, and because their fixed-size hidden state has limited capacity to retain information over many timesteps. Debugging: plot gradient norms across timesteps during BPTT -- gradients shrinking toward the earliest timesteps confirm vanishing gradients; test whether performance degrades sharply as sequence length increases, which points to memory-capacity limits; inspect hidden-state activations for saturation. Optimization: replace vanilla RNN cells with LSTM or GRU units, which use gating mechanisms to preserve gradients over long sequences, add attention mechanisms so the model can directly access distant timesteps, and use gradient clipping together with truncated backpropagation-through-time for stable training.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class AttentionLSTM(nn.Module):
    def __init__(self, vocab_size, embed_dim=64, hidden_dim=128, num_classes=2):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim)
```

```

# LSTM gates (input/forget/output) mitigate vanishing gradients vs vanilla RNN
self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True, bidirectional=True)
self.attn = nn.Linear(hidden_dim * 2, 1) # attention over all timesteps
self.fc = nn.Linear(hidden_dim * 2, num_classes)

def forward(self, x):
    emb = self.embed(x) # (batch, seq_len, embed_dim)
    outputs, _ = self.lstm(emb) # (batch, seq_len, hidden*2)
    attn_scores = self.attn(outputs).squeeze(-1) # (batch, seq_len)
    attn_weights = F.softmax(attn_scores, dim=1) # lets model directly access distant timesteps
    context = torch.sum(outputs * attn_weights.unsqueeze(-1), dim=1)
    return self.fc(context)

def train_step(model, optimizer, x, y, clip_value=5.0):
    optimizer.zero_grad()
    logits = model(x)
    loss = F.cross_entropy(logits, y)
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), clip_value) # guards against exploding grads
    optimizer.step()
    return loss.item()

def check_gradient_flow(model):
    # Debugging helper: print gradient norm per named parameter to spot vanishing gradients
    for name, p in model.named_parameters():
        if p.grad is not None:
            print(f"{name:30s} grad_norm={p.grad.norm().item():.6f}")

if __name__ == "__main__":
    torch.manual_seed(0)
    vocab_size, seq_len, batch_size = 500, 40, 32
    model = AttentionLSTM(vocab_size=vocab_size)
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

    x = torch.randint(0, vocab_size, (batch_size, seq_len))
    y = torch.randint(0, 2, (batch_size,))

    for epoch in range(5):
        loss = train_step(model, optimizer, x, y)
        print(f"Epoch {epoch} | loss={loss:.4f}")
    check_gradient_flow(model)

```